

ACO TSP Solver

Ant Colony Optimization for the Travelling Salesman Problem
Ant System on natID Dense Matrices with Real-Time Visualization

Armin Memišević (index 20016)

Faculty of Electrical Engineering (ETF), University of Sarajevo
Course: Numerical Optimization — Prof. Dr. Izudin Džafić

Academic Year 2025/26

Abstract

This document describes **ACO TSP Solver** (build target `tspaco`, version 1.1), a desktop C++ application that solves the **Travelling Salesman Problem (TSP)** with the **Ant System**, the original **Ant Colony Optimization (ACO)** algorithm of Dorigo and Stützle. Each problem instance consists of 15–25 cities drawn at random from a set of 99 real towns in Bosnia and Herzegovina with true geographic coordinates. The application builds the $n \times n$ distance matrix and the $n \times n$ pheromone matrix as `dense::DblMatrix` objects from the natID matrix library, and lets a colony of artificial ants construct tours by roulette-wheel selection driven by pheromone intensity (α) and inverse distance (β). After every iteration the trail is evaporated and reinforced in proportion to Q/L_k , using whole-matrix library operators for the evaporation and deposit-application steps. Every tour is checked for feasibility (a genuine permutation of all cities) before it is costed or allowed to deposit pheromone. The search runs on a background thread while an interactive GUI, built on the `natGUI` framework and adapted from the professor's `B_S03_Maps` example, renders the cities, the best tour, the pheromone trail as edge thickness, a live convergence chart with a nearest-neighbour baseline, and a statistics sidebar. The application additionally supports new random instances on demand, seeded reproducibility, CSV export of the run and of the full problem instance, side-by-side comparison of the last two completed runs, and an English/Bosnian user interface.

Contents

1	Introduction	4
1.1	What This Project Is	4
1.2	Why a Metaheuristic Instead of Exact Search	4
1.3	Project Goals	4
1.4	Relationship to the Original Proposal	4
1.5	Starting Point: <code>B_S03_Maps</code>	6
2	Logic: Mathematical Model	6
2.1	The TSP as an Optimization Problem	6
2.2	Problem Instance and Distance Model	6

2.3	Distance Matrix	7
2.4	Pheromone Matrix and Initialization	7
2.5	Probabilistic Tour Construction	7
2.6	Feasibility Check	7
2.7	Global Pheromone Update (Ant System)	8
2.8	Complete Algorithm	8
2.9	Nearest-Neighbour Baseline	8
2.10	Convergence Indicators	9
2.11	Randomness and Reproducibility	10
2.12	Complexity	10
3	Usage	10
3.1	Purpose of the Application	10
3.2	What It Shows	10
3.3	Building and Running	11
3.4	Typical Session	11
4	Features	11
4.1	ACO Parameters	11
4.2	Problem Instances	12
4.3	CSV Export	12
4.4	Run Comparison	13
4.5	Multi-Language Support	13
4.6	Persistence	13
4.7	Packaging and Continuous Integration	13
5	Software Architecture	13
5.1	Source Files	13
5.2	Threading Model	14
6	GUI Features	15
6.1	Menus and Toolbar	16
6.2	Parameter Bar	16
6.3	Map Canvas	16
6.4	Convergence Chart	16
6.5	Statistics Sidebar	17
6.6	Settings Dialog	17
6.7	Compare Dialog	18
6.8	Help Dialog	18

7	Verification: Does the Implementation Match the Proposal?	18
7.1	Control-Flow Audit	18
7.2	What “No Cheating” Means	19
7.3	Mathematical Verification	19
7.4	Independent Numerical Check	20
7.5	Known Limitations	20
7.6	Conclusion	21
8	Conclusions	22
8.1	What the Project Demonstrates	22
8.2	Problems It Solves	22
8.3	Lessons Learned	22
8.4	Possible Future Work	22
	References	23

1 Introduction

1.1 What This Project Is

ACO TSP Solver is the term project for the Numerical Optimization course. It implements a complete Ant Colony Optimization solver for the symmetric Travelling Salesman Problem: given n cities, find the shortest closed tour that visits each city exactly once and returns to its starting point.

The project deliberately implements the *basic* Ant System rather than one of its later refinements (Ant Colony System, MAX–MIN Ant System) and adds no local search. This keeps every number the application reports traceable to a single, textbook update rule: whatever tour the colony finds, it found through probabilistic construction and pheromone feedback alone.

1.2 Why a Metaheuristic Instead of Exact Search

The TSP is NP-hard. For n cities the number of distinct undirected tours is $(n-1)!/2$, which is 4.4×10^{10} for the smallest instance the application generates ($n = 15$) and 3.1×10^{23} for the largest ($n = 25$). Exhaustive enumeration is out of the question. The best general exact method, the Held–Karp dynamic program, runs in $O(n^2 2^n)$ time and $O(n 2^n)$ memory; at $n = 25$ that is on the order of 2×10^{10} operations over a table of roughly 8×10^8 entries (several gigabytes).

Ant Colony Optimization trades the guarantee of optimality for a controllable amount of computation: m ants, each building one tour in $O(n^2)$, per iteration. Section 7.4 shows that, with the default parameters, on generated instances small enough to solve exactly ($n \leq 20$) the colony’s best tour was on average within 0.7% of the true optimum.

1.3 Project Goals

- Formulate the TSP as a combinatorial minimization problem over permutations of n cities.
- Store the $n \times n$ distance matrix D and the $n \times n$ pheromone matrix τ as natID dense matrices, and perform evaporation and the deposit update directly on them.
- Implement probabilistic tour construction by roulette-wheel selection weighted by $\tau_{ij}^\alpha \eta_{ij}^\beta$, with all parameters constant during a run.
- Implement the Ant System global pheromone update: evaporation followed by a deposit of Q/L_k from every ant.
- Run the iteration loop on a background thread, adapted from the B_S03_Maps thread architecture, so the interface stays responsive.
- Visualize the search live: cities, current best tour, pheromone strength as edge thickness, a convergence curve (cost vs. iteration), and a sidebar with iteration count, best cost and runtime.

1.4 Relationship to the Original Proposal

The project proposal (*Ant Colony Optimization for the Traveling Salesman Problem*) named Dorigo & Stützle’s *Ant Colony Optimization* (MIT Press, 2004) as the theoretical source and the professor’s natID example B_S03_Maps as the starting point. The delivered implementation follows both. The table below lists every item of the proposal against what was delivered.

Item	Proposal	Delivered (+ additions)
Problem instance	15–25 randomly generated cities	15–25 cities <i>randomly selected</i> from 99 real Bosnian towns (<code>res/maps/Bosnia.xml</code>), with real latitude/longitude. + <i>New cities</i> action draws a fresh instance at any time.
Distance matrix	$N \times N$ natID dense matrix	<code>dense::DblMatrix _dist</code> , in kilometres.
Pheromone matrix	$N \times N$ natID dense matrix	<code>dense::DblMatrix _pheromone</code> , plus a separate <code>dense::DblMatrix _deltaPheromone</code> for $\Delta\tau$.
Tour construction	Roulette-wheel selection with α, β	Implemented (<code>Model::constructTour</code>). + Every tour validated as a Hamiltonian cycle (<code>isHamiltonianTour</code>).
Pheromone update	Evaporation + deposit $\propto 1/L$	Implemented as $\tau \leftarrow (1 - \rho)\tau$, $\Delta\tau = \sum_k Q/L_k$, $\tau \leftarrow \tau + \Delta\tau$.
Parameters	α, β, ρ , ants, iterations constant during a run	Implemented: read once when Start is pressed. + Editable between runs, range-clamped, persisted across restarts.
Background thread	Adapted from <code>B_S03_Maps</code>	Implemented (<code>std::thread</code> in <code>Model::findPath</code>).
Convergence curve	Best cost vs. iteration, live	Implemented. + Per-iteration best curve and a repetitive nearest-neighbour reference line.
Map canvas	Cities, best tour, pheromone as edge thickness	Implemented. + Two pheromone display modes (All edges / Threshold) and selectable colours.
Sidebar	Iteration, best cost, runtime	Implemented. + Iteration of best, nearest-neighbour baseline, signed improvement, cities, ants, seed, status.
Controls	Start / Stop / Reset	Implemented (menu and toolbar).
<i>Not in the proposal, additionally delivered:</i>		
Reproducibility	–	Seed field (0 = random); seed and city-draw index recorded in exports.
Analysis	–	CSV export (convergence series + full problem instance); comparison of the last two completed runs.
Usability	–	English/Bosnian UI, Settings dialog, in-app Help, light/dark colour defaults, sounds.
Distribution	–	GitHub Actions workflow building Windows, macOS (Apple Silicon, Intel) and Linux packages.

The only substantive deviation from the literal wording of the proposal is the instance itself: instead of synthetic random points, the cities are a random subset of real towns, so the distance matrix reflects actual geography. Tour lengths are therefore reported in kilometres.

1.5 Starting Point: B_S03_Maps

From the professor's example the project keeps the application skeleton (`Application`, `Main Window`, `MenuBar`, `ToolBar`), the `Town/Primitive` drawing classes, the XML map loading in `ViewMap`, the canvas animation loop, and the background thread with its completion callback. What was removed or replaced:

- The `Graph` adjacency structure and the `<Roads>` network are gone. ACO needs a complete graph; every pair of selected cities is directly connected by its air distance. With a random subset of towns, the original road network would also have survived only as a few disconnected edges.
- The breadth-/depth-first search in the model was replaced by the ACO engine (`Model::runACO`), and the source/goal town pair by a single *start marker* (a TSP tour has no separate goal).
- The convergence chart, statistics sidebar, compare and help dialogs, CSV export and all ACO parameter controls are new.

2 Logic: Mathematical Model

2.1 The TSP as an Optimization Problem

Let $V = \{1, \dots, n\}$ be the selected cities and $d_{ij} = d_{ji} \geq 0$ the distance between cities i and j . A tour is a cyclic permutation $\pi = (\pi_1, \dots, \pi_n)$ of V . The problem is

$$\min_{\pi \in S_n} f(\pi) = \sum_{i=1}^{n-1} d_{\pi_i \pi_{i+1}} + d_{\pi_n \pi_1} \quad \text{s.t. each city appears in } \pi \text{ exactly once.} \quad (1)$$

The decision space is discrete and the feasible set is exactly the set of Hamiltonian cycles of the complete graph on V . ACO is the solver for (1); the constraint is enforced both structurally (an ant can only move to an unvisited city) and explicitly, by the feasibility check of Section 2.6.

2.2 Problem Instance and Distance Model

`res/maps/Bosnia.xml` contains 99 towns (latitudes 42.71–45.18°N, longitudes 15.80–19.34°E). On every load, `Model::loadTowns()` shuffles the full list with a seeded Mersenne Twister, draws the instance size uniformly from $\{15, \dots, 25\}$, and keeps that many towns.

Distances use a local equirectangular approximation with fixed kilometres-per-degree factors taken from the map file (`ratioLat="111"`, `ratioLong="80"`; the latter is $111.3 \cos 44.2^\circ \approx 79.8$ at the country's mean latitude):

$$d_{ij} = \sqrt{(80(\lambda_i - \lambda_j))^2 + (111(\varphi_i - \varphi_j))^2} \text{ km}, \quad (2)$$

where λ is longitude and φ latitude in degrees. In code the computation goes through the normalized $[0, 1]$ display coordinates of each town, multiplied by the ratios scaled by the bounding-box span (`setDistanceRatios(ratioLat*dy, ratioLong*dx)`); the bounding-box offsets and display margins cancel, so the result is (2) up to single-precision rounding. Section 7.4 quantifies the approximation error against great-circle distance.

2.3 Distance Matrix

`Model::buildDistanceMatrix()` allocates `_dist` with `reserve(n, n, nullptr, true)`, clears it with `zeros()` (the storage may be reused from a larger previous instance), and fills the strict upper triangle, mirroring each value into the lower triangle. The diagonal stays zero.

The matrix is built once in `loadTowns()`, immediately after the towns are placed, so that the nearest-neighbour baseline (Section 2.9) is available before any run starts. `runACO()` rebuilds it once more at the start of every run on the worker thread; this costs $O(n^2)$ and guarantees the solver never operates on a stale matrix.

2.4 Pheromone Matrix and Initialization

`Model::initPheromone()` allocates `_pheromone` and `_deltaPheromone` with the same dimensions, zeros both, and sets every *off-diagonal* entry of τ to τ_0 :

$$\tau_{ij}^{(0)} = \begin{cases} \tau_0, & i \neq j \\ 0, & i = j \end{cases} \quad \tau_0 = 1.0. \quad (3)$$

The diagonal is left at zero on purpose. A self-loop is never a legal move, so τ_{ii} is never read during construction and never written by a deposit. Seeding it with τ_0 would not change the search, but it would decay under evaporation and distort the column sums used by the pheromone norm (Section 2.10).

2.5 Probabilistic Tour Construction

Each iteration launches m ants. Following the Ant System, each ant starts at a city drawn uniformly at random from V (`startPick(rng)`); starting all ants at one fixed city would not change which tours are reachable, but it would correlate their first decisions. An ant at city i with the set $\mathcal{N}_i^k$ of still-unvisited cities moves to $j \in \mathcal{N}_i^k$ with probability

$$p_{ij}^k = \frac{\tau_{ij}^\alpha \eta_{ij}^\beta}{\sum_{l \in \mathcal{N}_i^k} \tau_{il}^\alpha \eta_{il}^\beta}, \quad \eta_{ij} = \begin{cases} 1/d_{ij}, & d_{ij} > 10^{-6} \\ 10^6, & \text{otherwise.} \end{cases} \quad (4)$$

The guard on η prevents a division by zero for coincident cities. Selection is an explicit roulette wheel in `constructTour()`: the unnormalized weights $w_l = \tau_{il}^\alpha \eta_{il}^\beta$ are accumulated into Σ , a number $r \sim U[0, \Sigma)$ is drawn, and the first candidate whose running sum reaches r is chosen. If $\Sigma \leq 0$ (which can only happen if every candidate's weight underflows to zero), the next city is chosen uniformly among the candidates instead. τ_{ij} and d_{ij} are read directly from the two dense matrices through their manipulators.

2.6 Feasibility Check

`Model::isHamiltonianTour()` accepts a tour only if it has exactly n entries, every id is in $1..n$, and no id repeats. It is applied twice:

1. in `runACO()`, *before* a tour is costed, compared with the incumbent, or stored for the deposit step; a rejected ant leaves an empty slot and contributes to nothing;
2. in `depositPheromone()`, as a second line of defence (which also skips the empty slots).

The construction procedure cannot currently produce an infeasible tour. The check exists because the failure would be silent and severe: a short, infeasible tour is *cheaper* than any real

one, so it would become the reported best, be drawn on the map, and bias every subsequent ant through its deposit.

The tour length is $L_k = f(\pi^k)$ from (1), including the closing edge back to the first city (`Model::tourLength`).

2.7 Global Pheromone Update (Ant System)

After all ants of an iteration have finished, the trail is updated in three named steps that map one-to-one onto the Ant System rule:

$$\tau \leftarrow (1 - \rho) \tau \quad (5)$$

$$\Delta\tau_{ij} \leftarrow \sum_{k=1}^m \Delta\tau_{ij}^k, \quad \Delta\tau_{ij}^k = \begin{cases} Q/L_k, & (i, j) \in T^k \\ 0, & \text{otherwise} \end{cases} \quad (6)$$

$$\tau \leftarrow \tau + \Delta\tau \quad (7)$$

Step	Method	Operation on the <code>natID</code> matrix
(5)	<code>evaporatePheromone()</code>	<code>_pheromone *= 1 - rho</code>
(6)	<code>beginPheromoneDeposit()</code> , <code>depositPheromone()</code>	<code>_deltaPheromone.zeros()</code> , then per-edge <code>+= Q/L</code>
(7)	<code>applyPheromoneDeposit()</code>	<code>_pheromone += _deltaPheromone</code>

Together they equal $\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \sum_k \Delta\tau_{ij}^k$ as given by Dorigo and Stützle. Three implementation points matter:

- **Order.** Evaporation acts on the trail as it stood at the start of the iteration; this iteration's deposits are added afterwards and are not evaporated.
- **Accumulate, then apply.** All ants' contributions are summed into the separate matrix $\Delta\tau$ (zeroed at the start of each update) and added to τ in one operation. No ant sees another ant's deposit within the same iteration, and small deposits are summed at full precision before being added to a much larger trail.
- **Symmetry.** The TSP is symmetric, so a traversed edge receives the deposit in both $\Delta\tau_{ab}$ and $\Delta\tau_{ba}$, including the closing edge of the tour.

Evaporation (5) and application (7) are whole-matrix `dense::Matrix` operators. The deposit (6) is necessarily per-edge: a tour touches only n of the n^2 entries, and no whole-matrix operation expresses “add Q/L_k along this permutation”.

2.8 Complete Algorithm

The stored best tour is rotated (`std::rotate`) so that it begins at the city that carried the start marker when the run was started (the anchor is read once, at the start of `runACO()`). This is presentation only: a tour is a cycle, so rotation changes neither the tour nor its length, and the marker does not influence where ants start.

2.9 Nearest-Neighbour Baseline

A bare tour length is unanchored: whether 900 km is good depends entirely on which cities were drawn. `Model::computeNearestNeighbourTour()` therefore computes a *repetitive nearest-*

Algorithm 1 Ant System as implemented in `Model::runACO()`

```

1: build  $D$ ; set  $\tau_{ij} \leftarrow \tau_0$  for  $i \neq j$ ,  $\tau_{ii} \leftarrow 0$ ;  $L^* \leftarrow \infty$ 
2:  $\text{rng} \leftarrow \text{makeRNG}(\text{seed}, \text{stream } 1)$ 
3: for  $t = 1$  to  $N_{\text{iter}}$  do
4:   if Stop requested then break
5:   end if
6:    $L^{\text{iter}} \leftarrow \infty$ 
7:   for  $k = 1$  to  $m$  do
8:     if Stop requested then break
9:     end if
10:     $s \sim U\{1..n\}$ ;  $T^k \leftarrow \text{CONSTRUCTTOUR}(s)$  using (4)
11:    if not ISHAMILTONIAN( $T^k$ ) then continue
12:    end if
13:     $L_k \leftarrow f(T^k)$ ;  $L^{\text{iter}} \leftarrow \min(L^{\text{iter}}, L_k)$ 
14:    if  $L_k < L^*$  then
15:       $L^* \leftarrow L_k$ ;  $T^* \leftarrow T^k$  rotated to the start marker;  $t^* \leftarrow t$  ▷ under mutex
16:    end if
17:  end for
18:   $\tau \leftarrow (1 - \rho)\tau$ ;  $\Delta\tau \leftarrow 0$ ;  $\Delta\tau += Q/L_k$  along each feasible  $T^k$ ;  $\tau \leftarrow \tau + \Delta\tau$ 
19:  record  $L^*$ ,  $L^{\text{iter}}$ ,  $\|\tau\|_1$ ; publish a copy of  $\tau$  for drawing ▷ under mutex
20:  sleep sleepMS milliseconds ▷ animation speed
21: end for
22:  $\text{completed} \leftarrow \text{not Stop requested}$ 

```

neighbour tour for each instance: the greedy “always go to the closest unvisited city” construction is run from *every* starting city and the shortest of the n resulting tours is kept ($O(n^3)$, once per instance). By construction it is never longer than a nearest-neighbour tour from any single start city. It is used only for display and export:

$$\text{improvement} = 100 \cdot \frac{L_{\text{NN}} - L^*}{L_{\text{NN}}} \%, \quad (8)$$

shown *signed and unclamped*. A run stopped early or given too few ants or iterations can genuinely fail to beat the greedy tour, and the sidebar reports that rather than hiding it. The baseline belongs to the instance: it survives Reset and is recomputed only when new cities are drawn. It is never used to seed or bias the colony.

2.10 Convergence Indicators

Three series are appended together, under one lock, after each iteration’s pheromone update (skipped only if Stop arrived before any ant of that iteration had completed a tour):

- **Best-so-far** L_t^* – the shortest tour found in iterations $1..t$; monotonically non-increasing.
- **Iteration best** L_t^{iter} – the shortest tour built during iteration t alone. It is the noisy series: it shows the colony still exploring and settles towards the best-so-far curve as the trail sharpens.
- **Pheromone norm** $\|\tau\|_1$ – `dense::Matrix::getL1Norm()`, the maximum absolute column sum of τ after the update (exported only).

What the pheromone norm actually measures. Every feasible tour enters and leaves each city exactly once, and deposits are written symmetrically, so each ant adds exactly $2Q/L_k$ to

every column of τ . With the zero diagonal all columns start equal at $(n-1)\tau_0$, and they therefore stay equal:

$$\|\tau^{(t)}\|_1 = (1 - \rho) \|\tau^{(t-1)}\|_1 + 2 \sum_k \frac{Q}{L_k}, \quad \|\tau^{(0)}\|_1 = (n-1)\tau_0. \quad (9)$$

The norm is thus a deterministic function of the iteration’s tour lengths. It measures total trail mass per city, not how concentrated the trail is on a few edges. With the default parameters it *decreases* from $(n-1)\tau_0$ towards approximately $2mQ/(\rho\bar{L})$ (about 7 for tours of ≈ 1100 km). Recurrence (9) was confirmed numerically to machine precision (Section 7.4).

“Converged” status. The chart and sidebar label a run as converged when the best-so-far cost has not improved for 15 consecutive iterations (`cConvergencePatienceIterations`). This is a display heuristic only; the search always runs all N_{iter} iterations unless Stop is pressed.

2.11 Randomness and Reproducibility

All randomness comes from `std::mt19937` engines built by `Model::makeRNG(seed, stream)`:

- `seed = 0` means *random*: the engine is seeded from `std::random_device`;
- otherwise the engine is seeded with `seed + stream · 0x9E3779B9`, where even streams $(0, 2, 4, \dots)$ are the successive city draws and stream 1 is the ants’ decisions.

The city-draw counter advances on every draw, so with a fixed seed each press of *New cities* still produces a different instance, while relaunching the application with the same seed replays the same sequence of instances. A run is therefore identified by *two* numbers, the seed and the `city_draw_index`; both are written to every export. A seed typed mid-session affects the ants on the next Start and the cities only on the next *New cities*.

2.12 Complexity

Per iteration, tour construction costs $O(mn^2)$ (each of $n-1$ steps scans all candidates), feasibility and costing $O(mn)$, evaporation and deposit application $O(n^2)$, and the deposit $O(mn)$. With $n \leq 25$ and the default $m = 20$ the computational work per iteration is negligible; the wall-clock pace of a run is dominated by the deliberate inter-iteration delay used for animation (default 500 ms).

3 Usage

3.1 Purpose of the Application

ACO TSP Solver is a teaching and experimentation tool for Ant Colony Optimization. It is built to answer questions such as: how do α and β trade trail-following against greedy distance choices? Does a higher evaporation rate converge sooner, or only more noisily? How many ants and iterations are needed before the colony reliably beats a strong greedy tour? Because every run can be exported and the last two completed runs compared side by side, those questions can be answered with numbers rather than impressions.

3.2 What It Shows

While a run is in progress the application:

1. constructs m tours per iteration on a background thread and updates the pheromone matrix;
2. redraws the map in an animation loop that requests 60 fps: the pheromone layer (edge thickness $\propto$ trail strength), the cities, and the current best tour;
3. redraws the convergence chart and statistics sidebar in loops that request 10 fps, from thread-safe snapshots;
4. on completion, stops the animation, stores the run for comparison if it ran to its last iteration, and plays a short sound (if enabled).

3.3 Building and Running

The project builds with CMake (≥ 3.18) against the natID SDK, which must be installed at `~/natID.SDK` (`%USERPROFILE%\natID.SDK` on Windows). It links the `mainUtils`, `natGUI` and `MatrixLib` libraries; nothing outside the natID SDK is required (on Linux the SDK itself needs the GTK4, libadwaita and OpenAL development packages). The sources are in the `Implementation/` folder of the repository, which is the folder CMake or the IDE must be pointed at.

```
cmake -S Implementation -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
```

On macOS the project can also be generated for Xcode (`cmake -S Implementation -B build -G Xcode`, or the CMake GUI with the Xcode generator); Debug or Release is then selected in the Xcode scheme. The SDK redirects build output away from the build folder; with the command-line build above the executable is written to `~/natID.RAMDisk/Out/tspaco/Release/` (`tspaco.exe`, `tspaco.app` or `tspaco`).

3.4 Typical Session

1. Launch the application. A random instance of 15–25 cities is already drawn, and its nearest-neighbour baseline is shown in the sidebar.
2. Optionally adjust the parameters in the top bar (ants, iterations, α , β , evaporation rate, seed) and the animation speed.
3. Press **Start**. Watch the best tour change on the map, the pheromone trail thin out onto a few edges, and the two convergence curves approach each other.
4. Press **Stop** at any time, or let the run finish.
5. Change one parameter and press **Start** again on the same cities; then open **Compare** to see both completed runs side by side.
6. Use **Export run (CSV)** to save the run and its instance to the Desktop for offline analysis.
7. Press **New cities** to repeat the experiment on a different instance.

4 Features

4.1 ACO Parameters

All values entered in the top bar are clamped to the ranges below when Start is pressed, and a clamped value is written back into its field so the user sees what the run actually uses. The

same clamping is applied to values loaded from a previous session. Q and τ_0 are compile-time constants (`Constants.h`) and are not exposed in the interface.

Parameter	Symbol	Default	Allowed range	Notes
Number of ants	m	20	1 – 500	Tours built per iteration
Number of iterations	N_{iter}	100	1 – 100 000	Length of the run
Alpha (pheromone)	α	1.0	0 – 10	Weight of the trail in (4)
Beta (heuristic)	β	3.0	0 – 10	Weight of $1/d_{ij}$ in (4)
Evaporation rate	ρ	0.5	0.01 – 0.99	Fraction of trail removed per iteration
Deposit constant	Q	100	fixed	Not editable
Initial pheromone	τ_0	1.0	fixed	Not editable
Seed	–	0 (random)	0 – 2 147 483 647	Non-zero makes runs reproducible
Speed	–	500 ms delay	0 – 2000 ms	Delay between iterations; live during a run

The bounds are not cosmetic. $m \leq 0$ would reach `std::vector` construction on the worker thread with a negative size, and $\rho \notin (0, 1)$ would drive the trail negative, turn `std::pow` into NaN and hand a NaN bound to the random distribution.

4.2 Problem Instances

New cities reparses `Bosnia.xml`, draws a new random subset of 15–25 towns, rebuilds the distance matrix and baseline, repopulates the start-marker list, and clears all run state including the comparison history (costs from different instances are not comparable). *Reset* clears only the current run’s results and keeps the cities, the baseline and the comparison history.

4.3 CSV Export

Export run (CSV) writes two files to the user’s Desktop (falling back to the home folder, then the current folder) and reports their location in an alert:

- `aco_<YYYYMMDD_HHMMSS>_convergence.csv` – a single rectangular table with columns `iter`, `best_so_far_km`, `iter_best_km`, `pheromone_l1`, preceded by `#-comment` header lines. It loads directly into a spreadsheet, or into pandas with `read_csv(path, comment='#')`.
- `aco_<YYYYMMDD_HHMMSS>_instance.csv` – three blocks: the cities (`city_id`, `name`, `lat_deg`, `long_deg`, `x_norm`, `y_norm`); the best tour with the length of each leg, the last leg closing the cycle so that the leg column sums (up to rounding) to the reported cost; and the full $n \times n$ distance matrix in kilometres.

Both headers record the parameters, `cities`, `iterations_requested`, `iterations_run`, `completed` (0 if the run was stopped), `seed` (written as *random* – run is *NOT* reproducible when it is 0), `city_draw_index`, `nn_baseline_km`, `best_cost_km`, `best_found_at_iter`, `improvement_vs_nn_pct` and `runtime_s`. The files are written in the classic C locale (so a locale with a decimal comma cannot corrupt the columns) and start with a UTF-8 byte order mark (so names such as *Kotor Varoš* or *Goražde* display correctly in Excel). Export is refused while a run is in progress and when no run with at least one recorded iteration exists; a stopped run can be exported and is labelled `completed,0`.

4.4 Run Comparison

The application keeps the two most recent *completed* runs in memory. A run halted with Stop is deliberately not stored: its best cost reflects fewer iterations, and a table of raw kilometres would present that as a quality difference. *Compare* opens a dialog with the newer run, the older run and their difference, in three groups: the problem (cities, nearest-neighbour length), the parameters (ants, iterations, α , β , ρ , seed – without a difference column, since these are inputs, not results), and the result (best cost, improvement, iteration of best, converged yes/no, final-iteration best, mean iteration best, runtime). If fewer than two completed runs are stored, an alert states how many are.

4.5 Multi-Language Support

The interface is available in English and Bosnian. Both translation files (`res/tr/EN/main.xml`, `res/tr/BA/main.xml`) define the same 95 keys; standard strings such as OK, Cancel and Quit come from the natGUI common translations. The language is chosen in Settings and takes effect after a restart, which the application offers to perform. The body text of the Help dialog is English in both languages.

4.6 Persistence

Parameters, seed, animation speed, path colour, start marker, pheromone display mode and colour, the show-town-names and sound options, and the language are stored through the natGUI application properties (the registry on Windows, a property list on macOS, the settings store on Linux) and restored on the next launch.

4.7 Packaging and Continuous Integration

`.github/workflows/release-all.yml` installs the natID SDK from scratch, builds the Release configuration, and runs the SDK's `SetupCollector` with `Implementation/packaging/tspaco.xml` for Windows, macOS (Apple Silicon and Intel) and Linux, publishing the packages to a GitHub Release when a `v*` tag is pushed. The packaging configuration adds the `Matrix` package required by `MatrixLib`, and `packaging/GTK4.xml` corrects the GTK runtime paths on Windows.

5 Software Architecture

5.1 Source Files

Apart from `main.cpp`, all code is in headers under `Implementation/src/`.

File	Responsibility
<code>main.cpp</code>	Entry point; reads the persisted language and starts the application.
<code>Application.h</code>	natGUI application; creates the main window.
<code>MainWindow.h</code>	Menu and toolbar dispatch; guards (no reset, new cities, export, compare or close during a run); opens the dialogs.

File	Responsibility
MainView.h	Layout (parameter bar, map, sidebar, chart); reads, clamps and persists parameters; export and compare entry points.
MapModel.h	The ACO engine (Model): distance and pheromone matrices, tour construction, feasibility, pheromone update, baseline, worker thread, thread-safe snapshots (RunStats , ConvergenceSeries , RunRecord).
ViewMap.h	Map canvas: XML loading, pheromone layer, towns, best tour, run timer, completed-run history.
ViewConvergence.h	Convergence chart.
ViewStats.h	Statistics sidebar.
ViewCompare.h	Comparison of the last two completed runs.
DialogCompare.h	
ViewSettings.h	Settings dialog (language, toolbar labels, town names, sound, pheromone display).
DialogSettings.h	
ViewHelp.h	Static in-app help.
DialogHelp.h	
RunExport.h	All file I/O: the two CSV files.
Town.h	Drawable city and its display options.
Primitive.h	
MenuBar.h	Menu and toolbar definitions.
ToolBar.h	
Constants.h	Defaults and tuning constants; the 1-based city id type.
GraphType.h	

5.2 Threading Model

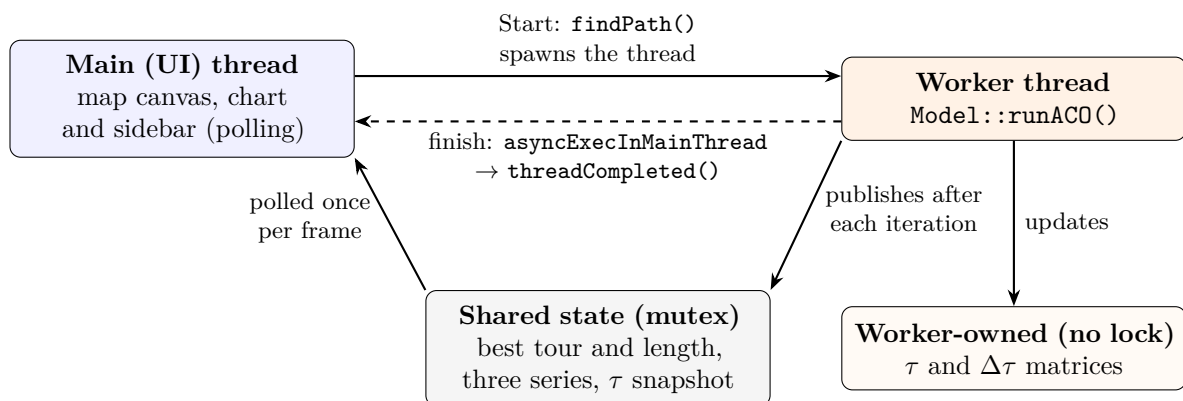


Figure 1: Division of work between the UI thread and the ACO worker thread.

- The worker owns τ and $\Delta\tau$ exclusively and updates them without locking. After each complete update it copies τ into a mutex-guarded snapshot, so the UI can never observe the trail mid-evaporation.
- The best tour, best length, the three history series, the pheromone snapshot and the view size are written only under **Model::_mutex**, and the UI thread reads them only under it. (The worker also compares against the best length without the lock, which is safe because

it is the only writer.) The chart's two curves are copied in one lock block, so they always have equal length. The sidebar reads the best length under the lock but the iteration counters as separate atomics, so for a single frame the counters can lag the best cost by one update.

- Counters and flags crossing threads outside the lock (`_iteration`, `_bestFoundAtIteration`, `_searching`, `_stopRequested`, `_tourFound`, `_ranToCompletion`, and the live `sleepMS` delay) are `std::atomic`.
- `gui::Shape` objects are created only on the main thread. The worker sets `_tourShape Dirty`; the tour polyline is rebuilt in the next `draw()`. The pheromone geometry ($O(n^2)$ lines) is rebuilt every 0.15s, or immediately after a resize, a new run, new cities or a settings change.
- Stop only sets `_stopRequested`, which the worker checks before each iteration and before each ant. Completion is delivered to the main thread with `gui::thread::asyncExecInMainThread`, where `ViewMap::threadCompleted()` joins the thread, stops the timer, stores the run if it completed, and refreshes the toolbar. The `Model` destructor also requests a stop and joins, and the window refuses to close while a run is active.

6 GUI Features

The main window (initially 1200×800) is a three-row grid: the parameter bar across the top; the map (15 of 20 columns) with the statistics sidebar beside it (5 columns); and the convergence chart across the full width, so that successive iterations are as far apart on screen as possible.



Figure 2: Main window during a run (16 cities, 20 ants, 40 iterations, $\alpha = 1$, $\beta = 3$, $\rho = 0.5$, random seed, Threshold pheromone mode), captured at iteration 25. The best tour (888.3 km, found at iteration 21) is already 3.7% shorter than the repetitive nearest-neighbour tour (922.6 km), which the chart draws as the horizontal reference line. The thin green lines are the pheromone edges above the threshold.

6.1 Menus and Toolbar

- **App** menu: Settings, Help, Quit.
- **Animation** menu: Start (checked while running), Reset, New cities, Export run (CSV), Compare last 2 runs.
- **Toolbar**: Settings, Start/Stop (icon, label and tooltip switch with the run state), Reset, New cities, Export run (CSV), Compare. Toolbar and menu items share action ids and are handled in one place (`MainWindow::onActionItem`).

Reset, New cities, Export and Compare are refused with an explanatory alert while a run is in progress, because they would either race the worker thread or capture a half-finished run.

6.2 Parameter Bar

Number of ants, number of iterations, Alpha, Beta, evaporation rate, a *Speed* slider (delay between iterations, applied live), a *Path color* picker (the best tour and the best-so-far curve), a *Start marker* combo box listing the current cities sorted by name, and the *Seed* field (0 = random).

6.3 Map Canvas

- **Pheromone layer** (drawn first): one line per city pair with thickness between 0.5 and 4.0 px, linearly scaled between the smallest and largest current trail value. In *Threshold* mode (the default) only edges whose trail exceeds $1.5\times$ the mean over all pairs are drawn, so the layer thins out by itself as the colony converges; *All edges* draws every pair.
- **Cities**: grey dots until the first tour exists; cities on the current best tour (after the first tour, every city) are drawn in the path colour, with names in dark blue (cyan in dark mode). A flag marks the start-marker city.
- **Best tour**: a closed polyline, 5.5 px, in the path colour.
- **Overlay** (bottom-left): frames per second and elapsed run time.

Default colours follow the operating system’s theme at start-up (e.g. dark red path and blue pheromone in light mode, orange path and green pheromone in dark mode). With sound enabled, a looping sound plays while the search runs and a short cheering sound plays when it finishes with a tour.

6.4 Convergence Chart

The proposal’s primary visualization. It plots, against iteration number $1..N_{\text{iter}}$:

- **Best so far** – thick line in the path colour, so it reads as the same object as the tour on the map;
- **Iteration best** – thin grey line;
- **Nearest neighbour** – a horizontal reference line labelled with its length.

The vertical range covers both curves *and* the baseline, with 6% padding. Including the baseline anchors the scale to a property of the instance: without it, an auto-scaled staircase fills the plot height whether the run improved by 40% or by 0.3%. A status line reports “Best at iter *a* (iter

$b/N)$ ” or, after 15 iterations without improvement, “Converged at iter a ”. The curves appear once two iterations have been recorded. Sample i (counting from 0) is plotted at $x = (i+1)/N_{\text{iter}}$, so the “0” tick means “before the first iteration”.

6.5 Statistics Sidebar

Row	Meaning
Iteration	Current iteration / requested iterations.
Best cost	Length of the best tour so far, in km (-- before the first tour).
Best found at	Iteration that produced the best tour.
Nearest neighbour	Repetitive nearest-neighbour tour length for this instance; available before Start.
Improvement	Signed percentage by which the best tour is shorter than the baseline.
Runtime	Wall-clock time since Start, including the animation delay between iterations.
Cities / Ants / Seed	Instance size, colony size, seed (“random” for 0).
Status	Idle; Running; Converged (running, 15 iterations without improvement); Finished (a run ended with a tour).

6.6 Settings Dialog

Opened from the toolbar or the App menu. It shows the current language and a language to use after restart; *Show labels below icons on main toolbar* (applied immediately); *Show town names*; *Play sound while searching*; *Pheromone display* (All edges / Threshold); and *Pheromone color*. Town names, sound and the pheromone options are populated from the values in force, and the language from the stored setting, so opening the dialog and pressing OK changes nothing by itself. The toolbar-label checkbox takes effect as soon as it is clicked. If a different language is selected, the application offers to restart when the dialog closes.

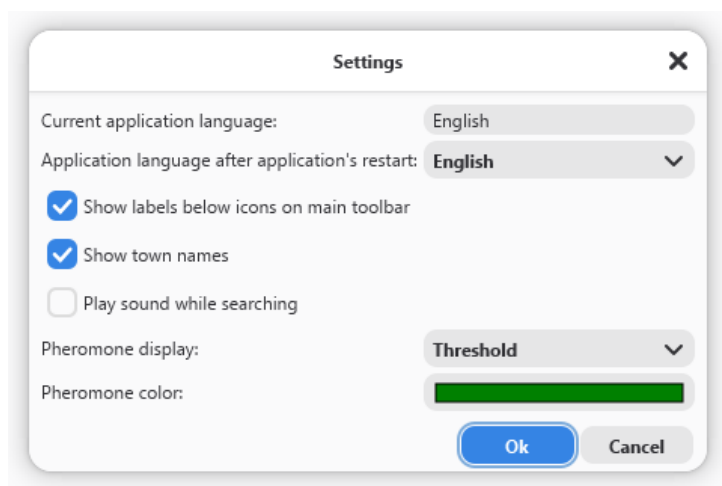


Figure 3: The Settings dialog.

6.7 Compare Dialog

A read-only dialog built from copies of the two stored run records, so a run started behind it cannot change its contents. Its contents are described in Section 4.4.

Comparison of the last two completed runs			
Metric	Newer run	Older run	Difference
Problem			
Cities:	16	16	--
Nearest neighbour:	884.8 km	884.8 km	--
Parameters			
Ants:	20	20	--
Number of iterations:	40	40	--
Alpha (pheromone):	1.00	1.00	--
Beta (heuristic):	5.00	3.00	--
Evaporation rate:	0.50	0.50	--
Seed:	random	random	--
Result			
Best cost:	863.5 km	863.5 km	+0.0 km
Improvement:	+2.4 %	+2.4 %	+0.0 %
Best found at:	6	4	+2
Converged:	yes	yes	--
Final iteration best:	863.5 km	863.5 km	+0.0 km
Mean iteration best:	874.8 km	871.1 km	+3.7 km
Runtime:	0.01 s	0.01 s	-0.00 s
Both runs were solved over the same set of cities.			

Figure 4: Comparison of two completed runs over the same 16 cities, differing only in β (5 in the newer run, 3 in the older). Both reach the same 863.5 km tour (2.4% below the nearest-neighbour tour); with $\beta = 5$ the colony finds it two iterations later and its mean iteration best is 3.7 km higher. Both runs were made with the speed slider at zero delay, so the runtime of 0.01 s is essentially the computational cost of the 40 iterations.

6.8 Help Dialog

A static page (App → Help) explaining the problem, the search, the map, the chart, the sidebar, the controls, the settings and reproducibility.

7 Verification: Does the Implementation Match the Proposal?

For this document the complete `src/` and `res/` trees were cross-checked against the proposal and against the Ant System as defined by Dorigo and Stützle. The central claim – every reported tour is the output of a genuine, live Ant System run, with no precomputed answers and no auxiliary optimizer – is confirmed by the control-flow audit below and by an independent numerical re-computation.

7.1 Control-Flow Audit

The path from pressing Start to a reported result is a single chain with no bypass:

```
MainWindow::onActionItem (Animation/Start)
```

```

-> MainView::startStop
-> readACOParamsFromUI          (clamp, persist, snapshot parameters)
-> ViewMap::start
  -> Model::findPath -> reset() -> std::thread
    -> Model::runACO
      -> buildDistanceMatrix, initPheromone          (dense::DblMatrix)
      -> for iter: for ant:
        constructTour (roulette, tau^alpha * eta^beta)
        isHamiltonianTour -> tourLength -> update best (mutex)
      -> evaporatePheromone          (_pheromone *= 1 - rho)
      -> beginPheromoneDeposit      (_deltaPheromone.zeros())
      -> depositPheromone x m        (Q / L_k, symmetric)
      -> applyPheromoneDeposit      (_pheromone += _deltaPheromone)
      -> getL1Norm, publishPheromoneSnapshot, append series (mutex)
    -> asyncExecInMainThread(ViewMap::threadCompleted)
    -> join, stop timer, captureCompletedRun, refresh toolbar

```

Every best tour shown on the map, plotted on the chart, listed in the sidebar, stored for comparison or exported passes through `constructTour` and `isHamiltonianTour` inside this chain.

7.2 What “No Cheating” Means

1. **No local search.** There is no 2-opt, 3-opt or Or-opt step; tours are exactly as the ants built them.
2. **No precomputed solutions.** There is no lookup table and no stored tour; each instance is drawn at run time.
3. **The baseline does not help the colony.** The nearest-neighbour tour is computed separately and used only for display and export; τ is initialized to the constant τ_0 , not from the greedy tour.
4. **No infeasible solutions.** Every tour is verified to be a permutation of all cities before it can be reported or reinforced.
5. **No elitism or trail limits.** Only the plain Ant System update is applied: no extra deposit on the best-so-far tour, no $\tau_{\min}/\tau_{\max}$ bounds.

7.3 Mathematical Verification

1. **Transition rule:** the weights are $\tau_{ij}^\alpha \eta_{ij}^\beta$ with $\eta = 1/d$, over unvisited cities only, sampled in proportion to weight – exactly (4).
2. **Update rule:** evaporation of the old trail, accumulation of Q/L_k over all feasible ants into $\Delta\tau$, then a single addition – exactly (5)–(7); deposits are symmetric and include the closing edge.
3. **Objective:** `tourLength` includes the return edge, matching (1); the export’s per-leg column sums to the reported cost up to rounding.
4. **Parameters constant:** the ACO fields of `Primitive::Options` are written only on the UI thread while no run is in progress (at start-up, by `readACOParamsFromUI()` on Start, and the seed also on New cities) and are only read by the worker.
5. **Matrices:** D , τ and $\Delta\tau$ are `dense::DblMatrix`; construction reads them through manipulators, and evaporation and deposit application are library operators.

7.4 Independent Numerical Check

To verify correctness against computed output rather than by inspection alone, the instance generator, the distance computation, the nearest-neighbour baseline and `runACO()` were re-implemented faithfully (same random-number streams, same operations and the same float/double types) in a standalone C++ program and run on the real `Bosnia.xml` with the default parameters ($m = 20$, 100 iterations, $\alpha = 1$, $\beta = 3$, $\rho = 0.5$, $Q = 100$, $\tau_0 = 1$). For every instance with $n \leq 20$ the true optimum was computed with the Held–Karp dynamic program.

Distance model. Over all 4755 town pairs more than 20 km apart, (2) differs from great-circle (haversine) distance by -1.8% to $+2.0\%$ (mean absolute error 0.4%, largest absolute error 4.7 km). Computing distances through the normalized display coordinates, as the application does, agrees with (2) to within 2×10^{-5} km on a test instance (single-precision rounding).

Solution quality (30 instances, seeds 1–30, first city draw):

Measure	Result
Instances with exact optimum available ($n \leq 20$)	18
ACO best tour equal to the optimum	10 of 18
Mean gap of ACO to the optimum	0.68%
Largest gap of ACO to the optimum	4.12%
Mean gap of repetitive nearest neighbour to the optimum	7.02%
Largest gap of repetitive nearest neighbour to the optimum	18.43%
Mean improvement of ACO over nearest neighbour (all 30)	+6.0%
Range of improvement over nearest neighbour	+0.0% to +16.4%

In no instance was the colony’s tour longer than the baseline; in three instances (all with the baseline already optimal or within 1.7% of it) the two were equal.

Pheromone norm. Recurrence (9) was checked with random tours on a 20-city instance for 50 iterations: the minimum and maximum column sums of τ agreed with each other and with the recurrence to nine decimal places at every iteration.

These figures characterize the algorithm, not a particular run of the application. The port was compiled with GNU libstdc++; for the reason given in Section 7.5, the application built with another standard library will draw different cities and make different ant decisions for the same seed.

7.5 Known Limitations

Stated so that no number in the application is read as more than it is:

- **Reproducibility is per toolchain.** `std::mt19937` is fully specified, but `std::shuffle`, `std::uniform_int_distribution` and `std::uniform_real_distribution` are implementation-defined. The same seed reproduces a run on the same platform and standard library (for example two Windows builds with MSVC), but not necessarily across MSVC, libc++ (macOS) and libstdc++ (Linux). The instance CSV compensates: it contains the full city list and distance matrix, so the problem itself can always be reconstructed.
- **Runtime is wall-clock time.** It includes the animation delay between iterations (500 ms by default, so 100 iterations take roughly 50 s). It measures the presentation, not the

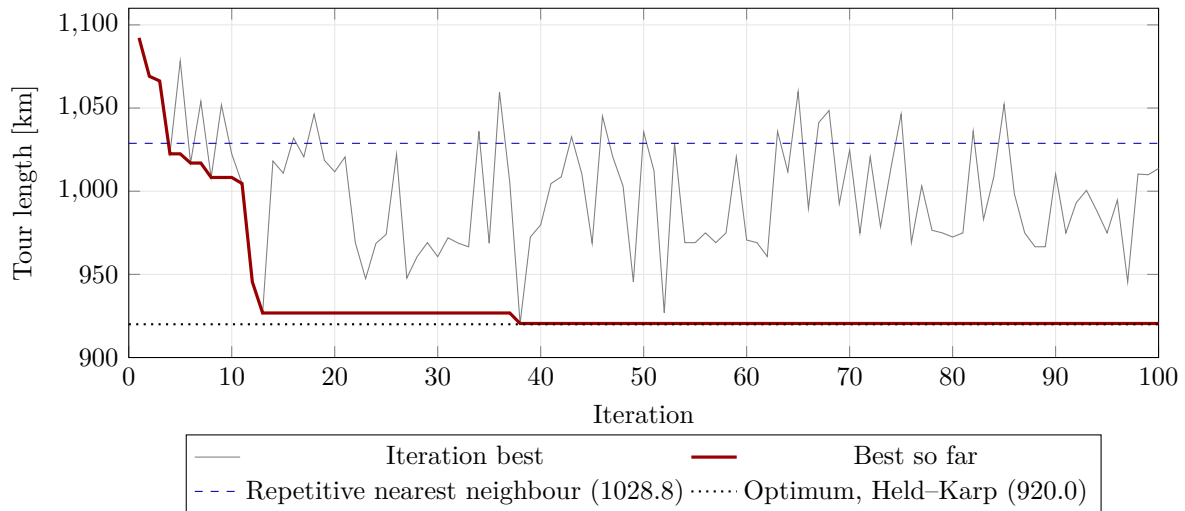


Figure 5: Convergence on one 19-city instance (seed 4) computed by the *reference port*, not a screenshot of the application. The best tour (920.5 km, found at iteration 38) is within 0.05% of the optimum; the nearest-neighbour tour is 11.8% above it. The same two curves and baseline are what the application’s convergence chart draws.

computational cost; runtime differences in the Compare dialog mostly reflect the speed slider.

- **pheromone₁₁ does not measure concentration.** By (9) it is determined by the tour lengths alone and is identical for every column; it records total trail mass per city.
- **Stopping mid-iteration** still applies that iteration’s pheromone update with the ants that had finished, and records the iteration; the worker also finishes the current inter-iteration delay (up to 2 s at the slowest speed) before the run ends. Such a run is marked `completed,0` in exports and is not stored for comparison.
- **Fixed τ_0 .** Dorigo and Stützle suggest $\tau_0 = m/C^{\text{nn}}$ for the Ant System; the application uses $\tau_0 = 1$. With $\rho = 0.5$ the initial trail falls below 1% after seven iterations, so the choice mainly affects how long the early iterations are dominated by the distance heuristic.
- **Distances are air distances** under a local flat-earth approximation (within $\pm 2\%$ of great-circle distance), not road distances.
- **“Show town names” off** shows names only for cities on the best tour. Since every tour contains every city, names are hidden only until the first tour exists.

7.6 Conclusion

The algorithm is a legitimate Ant System. Every reported tour is produced by probabilistic construction over the natID dense pheromone and distance matrices, validated as a Hamiltonian cycle, and reinforced by the textbook evaporation-and-deposit rule, with all parameters constant for the run. There is no local search, no precomputed solution, and no use of the greedy baseline inside the search. In the independent re-computation with the default parameters, the same algorithm reached the exact optimum in 10 of the 18 instances that could be solved exactly and stayed within 0.7% of it on average.

8 Conclusions

8.1 What the Project Demonstrates

ACO TSP Solver shows end to end how a population-based metaheuristic solves an NP-hard combinatorial problem with nothing more than two matrices and one feedback rule: ants read τ and D to build tours, the tours write back into τ in proportion to their quality, and evaporation keeps the colony from locking onto its first guesses. The live map makes that feedback visible – the pheromone layer starts uniform and collapses onto a handful of edges – while the convergence chart and the nearest-neighbour baseline turn “it seems to work” into a measurable claim.

8.2 Problems It Solves

- Provides a concrete, inspectable answer to “how does Ant Colony Optimization actually work”, down to individual matrix entries and random draws.
- Gives a sandbox for studying the effect of α , β , ρ , colony size and run length on real geographic instances, with side-by-side comparison of two runs.
- Produces analysis-ready data: the convergence series for plotting, and a self-contained instance file from which any reported tour length can be checked with a spreadsheet formula.
- Anchors every result to an external reference (a strong greedy tour), so a run that fails to beat it is reported as such.

8.3 Lessons Learned

- **A best-so-far curve alone proves little.** It is monotone by construction and fills any auto-scaled chart. The per-iteration best and a fixed baseline are what make convergence observable.
- **A solver must check its own constraints.** An infeasible tour is cheaper than a feasible one, so a construction bug would look like a spectacular result rather than a failure.
- **Instrumentation must be checked against the mathematics.** The pheromone L_1 norm looked like a convergence signal, but the Hamiltonian structure of every tour makes all column sums equal, so it cannot measure concentration.
- **Threading discipline is part of correctness.** A worker-owned matrix plus published snapshots, atomics for cross-thread flags, and main-thread-only graphics objects kept a live visualization from ever reading half-updated state.
- **Reproducibility needs more than a seed.** The city-draw index is required to identify an instance, and standard-library distributions differ between platforms.

8.4 Possible Future Work

- **Stronger ACO variants:** Ant Colony System (pseudo-random proportional rule, local update) and MAX-MIN Ant System (trail limits, best-ant deposit), selectable at run time for comparison.
- **Local search:** a 2-opt step on each constructed tour, which typically closes the remaining gap to the optimum.

- **A real concentration measure:** for example the mean entropy of the rows of the normalized transition probabilities, or the share of total trail on the current best tour's edges, to replace `pheromone_l1` in the export.
- **Platform-independent randomness:** hand-written uniform sampling on top of `std::mt19937`, so a seed reproduces a run identically on every platform.
- **Larger and external instances:** loading TSPLIB files and reporting the gap to their known optima.

References

- [1] M. Dorigo, T. Stützle, *Ant Colony Optimization*, MIT Press, Cambridge, MA, 2004.
- [2] A. Memišević, *Ant Colony Optimization for the Traveling Salesman Problem* – project proposal, ETF Sarajevo (Documents/NO_project_proposal_ArminMemisevic.pdf).
- [3] I. Džafić, *natID* framework and the B_S03_Maps example, <https://github.com/idzafic/natID>.
- [4] Project repository: https://github.com/arinn2206/ProjNO_TSPACO_Memisevic.